Davante

4

PROGRAMACIÓN

Programación modular

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Desarrollo modular 5

2.1. Diseño modular 6

/ 3. Caso práctico 1: “3, 2, 1... ¡Despegue!” 7

/ 4. Programa principal y subprogramas 8

/ 5. Visibilidad de variables en subprogramas 8

5.1. Variables locales 9

5.2. Variables globales 9

/ 6. Procedimientos y funciones 10

/ 7. Caso práctico 2: “Números comprendidos entre ¿?” 11

/ 8. Paso de parámetros a funciones 11

8.1. Paso de parámetros por valor 12

8.2. Paso de parámetros por referencia 13

8.3. Ejemplo de paso de parámetros por valor 13

8.4. Ejemplo de paso de parámetros por referencia 14

/ 9. Sobrecarga 15

9.1. Sobrecarga de operadores 15

9.2. Sobrecarga de métodos 15

ÍNDICE

/ 10. Funciones disponibles en los lenguajes de programación 16

/ 11. Resumen y resolución del caso práctico de la unidad 17

/ 12. Bibliografía 18

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Entender el concepto de programación modular.

Conocer el concepto de función y procedimiento junto a sus diferencias.

Definir la visibilidad de una variable.

Saber distinguir entre paso por valor y por referencia.

Conocer la sobrecarga de métodos.

/ 1. Introducción y contextualización práctica

En el mundo de la programación, la creación de software eficiente y mantenible es fundamental para el éxito de cualquier proyecto.

Sin embargo, al aumentar la extensión y la complejidad de los proyectos, es normal tener distintas partes, fragmentos de código que implementa una misma funcionalidad, generando distintos problemas, como puede ser:

- Duplicación de código: Fragmentos de código que se repitan en diferentes partes del programa. Esto no solo aumenta la cantidad de código que se necesita escribir, sino que también dificulta la actualización y el mantenimiento del software.

- Escasa reutilización de código: La falta de modularidad hace que sea difícil reutilizar el código en diferentes partes del programa o en proyectos diferentes. Esto conduce a una mayor duplicación de esfuerzos y a una menor eficiencia en el desarrollo de software.

Fig.1. Eco.

Para solventar estos problemas, es crucial entender y aplicar conceptos como la programación modular, el uso de funciones y procedimientos, la gestión de la visibilidad de las variables, el manejo adecuado del paso de parámetros y la sobrecarga de métodos.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Eco"

https://on.soundcloud.com/y7K6Tgd4dkbgINLM9

/ 2. Desarrollo modular

Los problemas a los que se enfrentan los programadores en su día a día tienen un grado de complejidad y a su vez son de gran tamaño, haciendo prácticamente imposible abordarlo de manera directa, siendo la solución su descomposición en problemas más pequeños.

La programación modular está basada en la metodología denominada '**divide y vencerás'**, es decir, en la división del programa en subprogramas, donde cada uno de ellos ejecuta una tarea independiente y se codifican por separado unos de otros, como muestra la siguiente imagen. El programa principal controla todo lo que sucede y los submódulos se ejecutan devolviendo el control al programa principal.

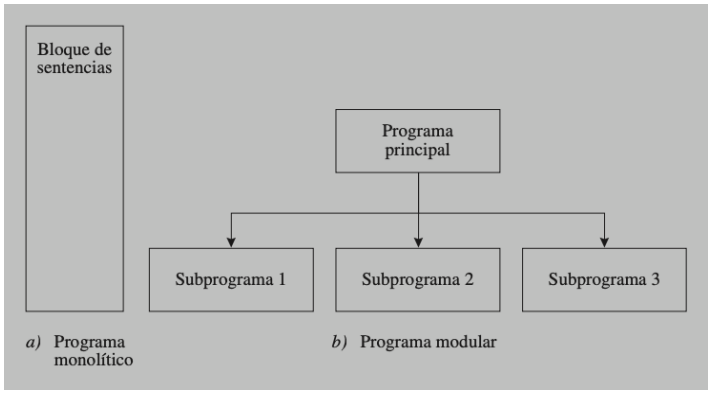

Fig.2. Programa monolítico vs. programa modular.

Gracias a la programación **modular()**, podemos dividir el software en distintos módulos que pueden realizar tareas diferentes, o, incluso, solucionar un tipo de problema en concreto. Podemos dividir cada módulo atendiendo al objetivo que tienen, por ejemplo: entrada, salida, manipulación de datos, etc.

Una ventaja de la programación modular es que **cada módulo puede ser desarrollado de forma independiente, mediante equipos independientes y probado de manera independiente**.

Sin tener en cuenta el número de módulos, existirá un programa principal que es el que se va a encargar de la ejecución y el control de flujo de cada uno de los módulos o sus programas.

Es por ello que un programa quedará formado por una serie de módulos, cada uno de los cuales realiza una parte concreta de la tarea total y serán analizados y desarrollados por separado. Por lo que la estructura de un programa modular consistirá en un módulo principal, el cual se encargará de llamar (o invocar) al resto de módulos o subprogramas, donde cada uno de ellos tendrá una estructura de caja negra, de forma que al facilitarle unos valores de entrada, devolverá unos valores de salida en función de los valores de entrada.

Podríamos decir que la programación modular es una de las mejores prácticas existentes, pues permite realizar componentes que se prueban previamente de manera independiente y facilita la reutilización del código.

¡! Sabías que...

La programación modular es ampliamente utilizada en soluciones actuales. Esta forma de desarrollo se considera una buena práctica que hay que seguir en desarrollos profesionales.

2.1. Diseño modular

La programación modular surge como mejora de la programación estructurada, con el objetivo de evitar la creación de programas de un único bloque o monolítico donde solo existe un programa principal de tamaño desmesurado, y así mejorar el mantenimiento de los mismos.

Un diseño modular descendente (top-down) consiste en una serie de descomposiciones sucesivas del algoritmo inicial denominadas módulos. Se trata, por tanto, de **unidades** de código **claramente definidas** y manejables, con **interfaces modulares**, creadas para ejecutar una **tarea específica**, que son compiladas (o interpretadas) de forma **independiente**.

Gracias al diseño descendente es posible crear programas desde una especificación de alto nivel a bajo nivel. Normalmente, comenzamos desgranando el problema en modelos más pequeños que tendrán una funcionalidad bastante concreta. Si enlazamos este proceso con el diseño modular, podemos ver que encajan a la perfección. Utilizando el diseño descendente, es posible diseñar programas muy complejos con un menor esfuerzo y con cierta seguridad de que podemos generar componentes que pueden ser reutilizables en el futuro.

Recuerda...

El diseño descendente se centra en generar una solución a partir de la división continua de un problema en subproblemas de fácil resolución. Gracias a este tipo de diseño es posible afrontar problemas más complejos.

Las ventajas que ofrece este tipo de programación son:

- Facilita el desarrollo de programas en equipo.
- Permite la reutilización de módulos, reduciendo el tiempo de desarrollo.
- Simplifica el diseño y ejecución de las pruebas.
- Reducción de los costes de mantenimiento.
- Mejora la claridad del diseño y de la documentación.
- Se reduce el número de bugs en el programa al reutilizarse módulos ya probados anteriormente.

Esto nos podría llevar a pensar que la mejor estrategia es dividir el software en el mayor número de módulos posibles, no obstante, como se puede observar en la siguiente figura, esto provoca que el esfuerzo necesario para desarrollar y mantener las interfaces por las que se comunican se incremente. Es por ello que es necesario buscar un equilibrio entre ambos esfuerzos.

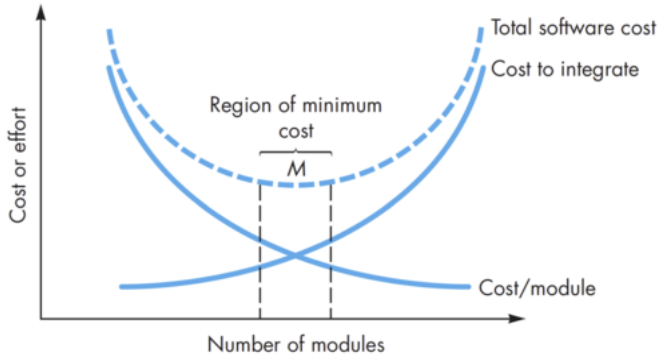

Fig.3. Coste de modularizar.

/ 3. Caso práctico 1: “3, 2, 1... ¡Despegue!”

Planteamiento: La programación modular es una técnica de desarrollo software que divide un programa en partes más pequeñas y manejables, conocidas como módulos. Cada módulo tiene una función específica y puede ser desarrollado, probado y mantenido de manera independiente, que facilita entre otras cosas la mantenibilidad y la reutilización de código.

Nudo: En el código que se presenta a continuación, se ejecuta un código que muestra una cuenta atrás hasta en tres ocasiones. Haciendo uso del diseño modular, implementa una solución que permita reutilizar el código mostrado en la figura para generar la cuenta atrás.

```java
public class CuentaAtras {
    public static void main(String[] args) {

        /*
        otro código
        */

        for (int i = 3; i > 0; i--) {
            System.out.println(i);
        }
        System.out.println("¡Despegue!");

        /*
        otro código
        */

        for (int i = 3; i > 0; i--) {
            System.out.println(i);
        }
        System.out.println("¡Despegue!");

        /*
        otro código
        */

        for (int i = 3; i > 0; i--) {
            System.out.println(i);
        }
        System.out.println("¡Despegue!");
    }
}
```

Fig.4. Programa monolítico.

Desenlace: Se ha creado un módulo ‘despegue’ que se encarga de generar la cuenta atrás y realizar la impresión como se muestra en la siguiente imagen.

```java
public class CuentaAtras {
    public static void main(String[] args) {

        /*
        otro código
        */

        despegue();

        /*
        otro código
        */

        despegue();

        /*
        otro código
        */

        despegue();
    }
}

public static void despegue(){
        for (int i = 3; i > 0; i--) {
            System.out.println(i);
        }
        System.out.println("¡Despegue!");
}
```

Fig.5. Programa monolítico.

/ 4. Programa principal y subprogramas

Dado un problema determinado en el que se pide realizar diez ordenaciones, podemos afrontarlo de dos maneras:

- En el caso de un **programa monolítico**, se debería repetir diez veces el mismo código desarrollo dentro del programa principal.
- En el caso de ser un **programa modular**, se desarrollaría un subprograma encargado de realizar la ordenación y se 'invocaría' diez veces desde el programa principal.

Un programa principal describe la solución completa del problema y consta, principalmente, de llamadas a subprogramas. Estas llamadas son indicaciones al procesador de que debe continuar la ejecución del programa en el subprograma llamado, regresando al punto de partida una vez lo haya concluido. El programa principal contendrá pocas líneas, y en él se verán claramente los diferentes pasos del proceso.

La estructura de un subprograma es básicamente la de un programa, con alguna diferencia en el encabezamiento y la finalización. Su función es resolver de modo independiente una parte del problema. Es importante que realice una función concreta en el contexto del problema. Un subprograma es ejecutado por el procesador exclusivamente al ser llamado por el programa principal o por otro subprograma.

Si el subprograma figura en el mismo archivo fuente que el principal, se le denomina **interno**. Por el contrario, si el subprograma figura físicamente separado del programa principal en otro archivo fuente, se le denomina **externo**.

Pueden ser compilados separadamente, incluso haber sido codificados en un lenguaje de programación distinto al principal. Generalmente, se enlazan con el programa principal en la fase de montaje o de enlazado (*linkage*), cuando ya han sido compilados.

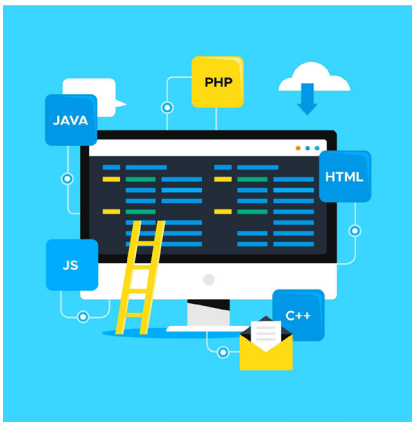

Fig.6. La programación modular puede requerir diferentes tecnologías.

Estos bloques o subprogramas se pueden denominar de distintas maneras en los lenguajes no orientados a objetos:

- Procedimientos.
- Funciones.

Sin embargo, en el caso de los lenguajes orientados a objetos, se denominan **métodos**.

Audio 1. "Modularización del código fuente y diseño descendente"
https://on.soundcloud.com/9kC3oFCxjguSUAvFA

/ 5. Visibilidad de variables en subprogramas

Los datos que se transmiten al subprograma reciben el nombre de **parámetros**. Cada parámetro se comporta como una variable, por lo que va a tener asociado un tipo de datos determinado que será manipulado correspondientemente dentro del subprograma. Cada subprograma puede tener en su interior **variables locales**, internas al subprograma, es decir, no son visibles desde el programa principal u otros subprogramas.

Las variables que están dentro del cuerpo de un subprograma no pueden ser accedidas desde fuera, es decir, están en un ámbito de visibilidad totalmente diferente. Sin embargo, es posible que los subprogramas sí puedan acceder a **variables globales** y pueden utilizarlas internamente dentro de su ámbito de visibilidad en conjunto con sus variables locales.

5.1. Variables locales

Las variables locales **solo existen en un ámbito determinado del programa**, por ejemplo, en un subprograma o en un bloque de sentencias.

Según esta regla, las variables declaradas en un subprograma son locales, no conociéndose fuera de él. Así, si dentro del subprograma se declara una variable con el mismo nombre que otra de otro subprograma, no hay ningún problema, pues las variables son distintas, correspondiendo cada una a áreas de memoria diferentes.

5.2. Variables globales

Las variables globales son aquellas que **se pueden acceder desde cualquier parte del programa**, ya que son declaradas en el programa principal. Esto significa que pueden ser utilizadas en cualquier módulo o subprograma.

Sin embargo, esto puede causar problemas porque cualquier parte del programa puede cambiar el valor de la variable sin control. Si luego es necesario usar la variable en otra parte del programa con su valor original, se obtendrá un error. Es fácil encontrar dónde ocurre el error, pero **no es tan fácil encontrar de dónde proviene** (es decir, dónde se modificó originalmente la variable). Estos efectos secundarios pueden causar errores difíciles de rastrear y localizar. Por esta razón, se recomienda evitar el uso de variables globales.

En la siguiente figura se pueden observar ambos ámbitos de una variable.

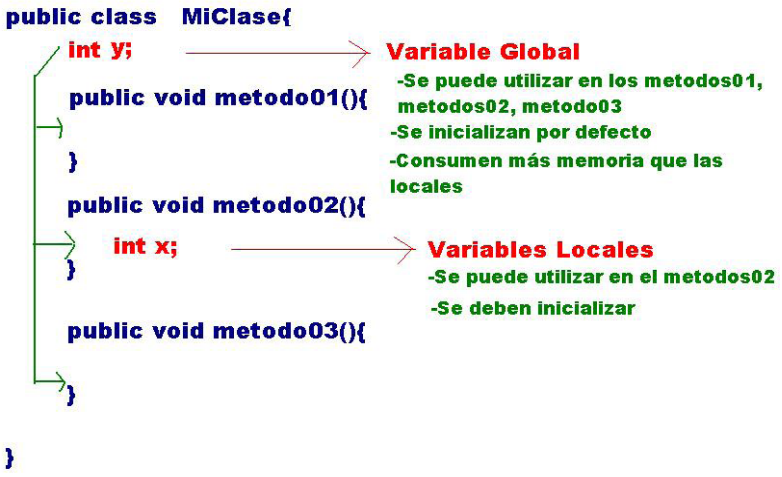

Fig.7. Ámbito de una variable.

/ 6. Procedimientos y funciones

Dentro de los subprogramas, podemos diferenciar dos grandes grupos: procedimientos y funciones. Se dice que los **procedimientos** son aquellas unidades algorítmicas que permiten realizar transformaciones en los estados en los que se encuentra el algoritmo. Mediante la utilización de los parámetros, los procedimientos pueden generar un nuevo estado, dando lugar a parámetros de salida. También es posible cambiar el estado modificando los parámetros de entrada/salida.

Recuerda...

Los procedimientos y las funciones son subprogramas que permiten reutilizar fragmentos de código de una forma más sencilla, evitando la duplicación del código y favoreciendo la programación estructurada.

En el caso de las **funciones**, podemos establecer una relación directa con su equivalente en el ámbito de las matemáticas. Una función permite, a partir de los valores de entrada, generar un resultado. Es el equivalente a tener una función matemática en el que se pasa un valor de X y se genera un valor Y como resultado. Podríamos decir que la única diferencia entre una función matemática y una función en programación es que la función en programación debe ser especificada paso a paso, es decir, mediante un algoritmo.

Aunque algunos lenguajes de programación permiten definir diferentes palabras claves para indicar si vamos a utilizar un procedimiento o una función, existen lenguajes que no utilizan ninguna palabra clave para diferenciar entre ambos elementos.

En este caso, la única diferencia que vamos a tener es el tipo de datos que devuelve cada una de ellas. En el caso de los procedimientos, no se devuelve ningún valor; mientras que en el caso de las funciones, podemos devolver un valor determinado.

Normalmente, se permite la definición de funciones mediante la especificación del tipo de dato devuelto en la cabecera de la función.

Tanto para la utilización del procedimiento como de funciones, podemos tener una lista de parámetros formales que cuando va a ser ejecutada, tomará el valor correspondiente a la ejecución, es decir, los valores que proporcione el llamador. Existen diferencias entre la utilización de procedimientos y funciones que deben tenerse en consideración para poder utilizar el que más nos convenga en cada caso.

Fig.8. Creación de funciones y procedimientos.

Vídeo 1. "Utilización de procedimientos y funciones"

https://vimeo.com/950698522/853d24d6fc

/ 7. Caso práctico 2: "Números comprendidos entre ¿?"

Planteamiento: El uso de funciones (o métodos, como se denominan en Java) es fundamental en la programación por varias razones, como son la reutilización de código y la descomposición del problema principal en problemas más pequeños.

Nudo: Se pide crear una función denominada NumersEntre en Java que muestre por pantalla los números comprendidos entre dos números pedidos por teclado desde el main.

Desenlace: Una posible solución de implementación sería la que se muestra en la siguiente figura:

```java
import java.util.Scanner;

public class NumerosEntre {

    // Función que muestra todos los números comprendidos entre dos enteros
    public static void mostrarNumerosEntre(int num1, int num2) {
        if (num1 > num2) {
            // Si num1 es mayor que num2, intercambiamos los valores
            int temp = num1;
            num1 = num2;
            num2 = temp;
        }
        // Imprimimos los números comprendidos entre num1 y num2
        for (int i = num1 + 1; i < num2; i++) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Pedimos los dos números al usuario
        System.out.print("Introduce el primer número: ");
        int numero1 = scanner.nextInt();

        System.out.print("Introduce el segundo número: ");
        int numero2 = scanner.nextInt();

        // Llamamos a la función para mostrar los números entre los dos enteros
        System.out.println("Números comprendidos entre " + numero1 + " y " + numero2 + ":");
        mostrarNumerosEntre(numero1, numero2);

    }
}
```

Fig.9. Creación de función NumerosEntre.

La función NumerosEntre toma dos enteros como parámetros y muestra todos los números comprendidos entre ellos. Para ello, si num1 es mayor que num2, intercambia sus valores para asegurarse de que num1 sea menor que num2. Después de intercambiar los números, si fuese necesario, utiliza un bucle for para imprimir los números comprendidos entre num1 y num2 (excluyendo num1 y num2).

/ 8. Paso de parámetros a funciones

El paso de parámetros a una función puede ser uno de los aspectos más tediosos a la hora de trabajar con funciones. Aunque no ocurre en Java, aquellos lenguajes que permitan un manejo exhaustivo de la memoria por parte del programador pueden introducir mecanismos que compliquen el paso de parámetros a funciones.

A la hora de trabajar con funciones, es necesario diferenciar entre los parámetros formales y los parámetros reales de la función, como se muestra en la siguiente imagen:

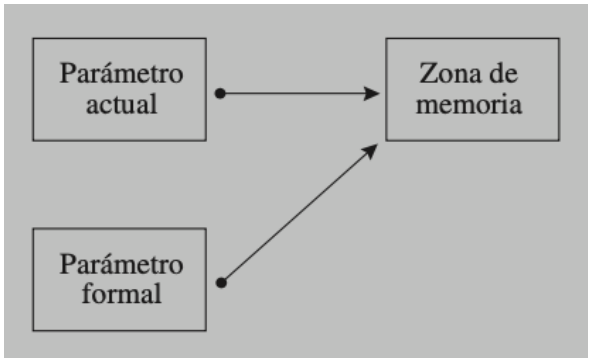

Fig.10. Paso de zona de memoria.

Los **parámetros formales** hacen referencia a aquellos parámetros que están especificados en el prototipo de la función, donde se especifica el tipo de cada uno de los parámetros. Uniendo todos los parámetros que tienen lugar en la definición de la función, creamos la lista de parámetros formales que utiliza dicha función.

Para poder utilizar una función dentro de cualquier módulo de nuestro programa o bien fuera de nuestros componentes, es necesario conocer la lista de parámetros, así como el nombre de la función. Si no es necesario trabajar con el valor devuelto, se puede obviar el tipo de retorno de la función, ya que no es 100% necesario.

Los **parámetros reales** de la función son aquellos que se utilizan durante la invocación a la función y que toman valor durante el flujo de ejecución. Estos parámetros deben coincidir en tipo y orden con los parámetros especificados en el prototipo de la función.

Es necesario especificar los parámetros en orden según se ha establecido en la lista de parámetros formales de manear que se permite establecer los parámetros mediante el nombre que tenga cada uno en la lista formal de parámetros.

El mecanismo por el que al invocar una función desde el programa principal se hace un mapeo entre parámetros reales y parámetros formales se conoce como paso de parámetros. Existen diferentes formas de pasar los parámetros a una función que van a depender del lenguaje de programación que estemos utilizando.

Podemos distinguir dos tipos de paso de parámetros:

- Paso de parámetros por **valor**.
- Paso de parámetros por **referencia**.

8.1. Paso de parámetros por valor

Mediante este paso de parámetros se establece que, durante el proceso de compilación, la función va a recibir una copia de los valores reales que tengan en tiempo de ejecución los parámetros reales con los que se llame a la función. Cuando se pasan los parámetros a una función por valor, si se realiza un cambio en una variable local, el valor solo afecta en el ámbito de visibilidad de la función y no tiene efecto fuera de este ámbito de visibilidad. Como se puede deducir, cuando se utiliza el paso de parámetros por valor, **la modificación de la variable que recibe el valor**, es decir, aquella que está en la lista de parámetros, **no afecta a la variable que realiza la invocación**. Esto es así porque durante el proceso de compilación se ha realizado una copia real del valor entre ambas variables.

Los argumentos de tipo primitivo se pasan por valor a los métodos. Esto significa que cuando pasas un argumento primitivo a un método, se copia el valor del argumento en el parámetro del método. Cualquier modificación realizada en el parámetro dentro del método no afectará al valor original del argumento fuera del método.

8.2. Paso de parámetros por referencia

Cuando utilizamos una función que permite un paso de parámetros por referencia, **los cambios que realicemos sobre la variable receptora van a afectar al valor de la variable que se utilice en la llamada formal de la función**. Para poder implementar este tipo de funcionamiento, es necesario emplear el uso de objetos que veremos más adelante. Al pasar un objeto como argumento a un método en Java, se está pasando la referencia al objeto, no el objeto en sí mismo. Sin embargo, es crucial entender que esta referencia es un valor, no la propia instancia del objeto. Por lo tanto, se está pasando una copia de la referencia al objeto, no el objeto en sí.

Aunque en Java se emplea el paso de parámetros por referencia al usar objetos, no soporta punteros directamente en el lenguaje, lo que significa que no se pueden manipular direcciones de memoria de forma explícita. Esto es parte del diseño de Java para asegurar la seguridad y simplicidad del manejo de memoria.

La siguiente imagen muestra un ejemplo de ambos tipos de paso de parámetros.

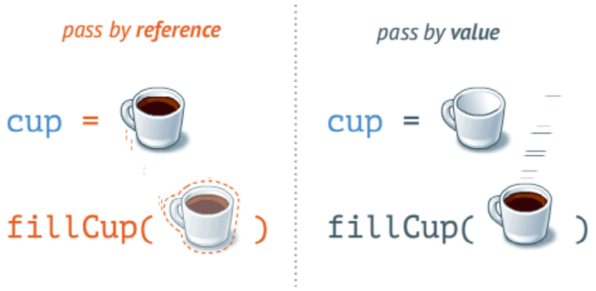

Fig.11. Diferencia entre paso por valor y referencia.

8.3. Ejemplo de paso de parámetros por valor

A continuación, se muestra un ejemplo (figura) de paso de parámetros por valor en el que se puede observar cómo funciona este tipo de paso de parámetros.

```
PROGRAMA

public static int test(int mivalor) {
        Imprimir(" Caso2 = " +  mivalor);
        mivalor=799;
        Imprimir("Caso3 = " + mivalor);
return mivalor;
}

public static void main(String [] args) {
        int num = 200;
        Imprimir("Caso1 = " + num);
        int resultado = test(num);
        Imprimir("Caso4 = " + num);
Imprimir("Caso5= " + resultado);
}

END PROGRAMA
```

Fig.12. Código de paso por valor.

En el código mostrado anteriormente, se puede observar que se ha creado una función que acepta un parámetro de tipo entero y realiza una modificación sobre su valor. Este tipo de paso de parámetros realiza una copia local del valor de la variable recibida sin afectar al valor de la variable con la que se realiza la invocación.

A continuación, vamos a analizar **qué imprime el algoritmo en cada situación**:

- **Caso1**: Este valor se imprime antes de llamar a la función, por lo que mostrará por pantalla 200. Este es el valor con el que está inicializada la variable.
- **Caso2**: Este valor se imprime dentro del cuerpo de la función, siendo también 200.
- **Caso3**: En este punto se realiza una modificación local, mostrando por pantalla 799.

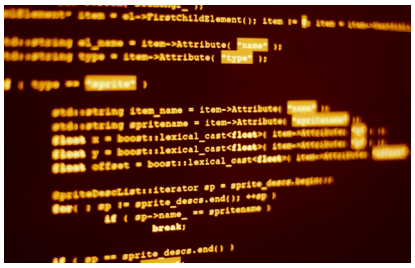

Fig.13. Paso de parámetros por valor.

- **Caso4**: Al realizarse un paso de parámetros por valor, no se transmite el valor interno a la función fuera de su ámbito de visibilidad. Se muestra por pantalla 200.
- **Caso5**: En este caso, se asigna el valor devuelto de la función a una variable, mostrando por pantalla 799.

8.4. Ejemplo de paso de parámetros por referencia

A continuación, se muestra un ejemplo (figura) de paso de parámetros por referencia en el que se puede observar cómo funciona este tipo de paso de parámetros.

```
PROGRAMA

public static void test(int []mivalor) {
        Imprimir(" Caso2 = " +  mivalor[0]);
        mivalor[0]=799;
        Imprimir("Caso3 = " + mivalor[0]);
}

public static void main(String [] args) {
        int [] num = {200,100};
        Imprimir("Caso1 = " + num[0]);
        test(num);
        Imprimir("Caso4 = " + num[0]);
Imprimir("Caso5= " + resultado[1]);
}

END PROGRAMA
```

Fig.14. Código de paso por referencia.

En el ejemplo anterior se puede observar cómo se ha creado una función que admite una variable que apunta a un valor entero (int []mivalor). En este código se está utilizando el operador [], que permiten trabajar con posiciones de memoria y valores de las variables. En el cuerpo de nuestro programa se realiza la llamada a la función **test**, utilizando el operador [], que proporciona la dirección de memoria de la variable **num**.

Al estar trabajando con operadores de matrices, se puede deducir que el paso de parámetros, en este caso, es por referencia. Analicemos cada una de las situaciones que se imprime por pantalla:

- **Caso1 y Caso2**: Se comporta exactamente igual que en el paso de parámetros por valor.
- **Caso3**: En este caso se realiza una modificación local, mostrando por pantalla 799. Para poder realizar la modificación, se modifica la posición de memoria [0].
- **Caso4**: Al hacerse un paso de parámetros por referencia, se ha modificado el valor interno de la función fuera de su ámbito de visibilidad. Se muestra por pantalla 799.

Fig.15. Paso de parámetros por referencia.

- **Caso5**: En este caso se muestra el valor de la segunda posición de la función 100.

/ 9. Sobrecarga

El concepto de sobrecarga se utiliza a menudo en programación para dar lugar a **elementos que tienen el mismo nombre, pero pueden trabajar con diferentes tipos de datos**. En ocasiones, la sobrecarga requiere un tipo de compilación dinámica porque no somos capaces de definir el tipo de datos con el que vamos a trabajar en tiempo de compilación. Desde el punto de vista de las funciones y los operadores es posible establecer diferentes tipos de sobrecargas que permiten flexibilizar el lenguaje de programación con el que estamos trabajando.

9.1. Sobrecarga de operadores

Se habla de sobrecarga de operadores **cuando el programador es capaz definir operadores con un nombre similar a los propuestos por el del lenguaje de programación**.

En el mundo real, podemos hacer referencia a situaciones en las que se puede dar el concepto de sobrecarga sin que seamos conscientes de ello. Por ejemplo, cuando operamos aritméticamente variables reales y enteras con los mismos operadores, podemos decir que el operador que estamos utilizando está sobrecargado. Además, podemos dar el significado que estimamos oportuno a cada uno de los operadores que tengamos definido en nuestro lenguaje. Es importante tener en cuenta que la sobrecarga de operadores va a estar limitada por el lenguaje de programación y tenemos que adaptarnos a las reglas que nos ponga en cada momento.

9.2. Sobrecarga de métodos

Al igual que sucede con la sobrecarga de operadores, los lenguajes de programación permiten establecer diferentes mecanismos para sobrecargar funciones que pertenezcan al lenguaje o bien hayan sido definidas por el programador. La sobrecarga de funciones nos permite **definir funciones con el mismo nombre que puedan trabajar con diferentes tipos de datos**. Cuando realicemos la invocación a la función, será el compilador el encargado de resolver qué función hay que invocar, teniendo en cuenta el tipo de datos que se especifica en la lista de parámetros. De esta forma, podemos tener la misma función definida con diferentes tipos de datos. Es importante destacar que, aunque dos funciones reciban el mismo nombre, su implementación puede ser completamente diferente. Debemos tener en cuenta que lo que estamos haciendo es mantener la semántica entre diferentes tipos de datos.

La siguiente imagen muestra un ejemplo práctico de sobrecarga de métodos.

```
public static void main(String[] args) {
    int valor;
    valor = suma(5);
    valor = suma(5,5);
}

public static int suma(int valor1){
    return valor1 + 5;
}

public static int suma(int valor1, int valor2){
    return valor1 + valor2;
}
```

Fig.16. Sobrecarga de métodos.

/ 10. Funciones disponibles en los lenguajes de programación

Al igual que sucede con los tipos de datos, donde los lenguajes de programación nos ofrecen un conjunto de tipos de datos primitivos inherentes al lenguaje de programación, podemos extender este comportamiento a las funciones. Los lenguajes de programación más modernos nos permiten utilizar un conjunto muy amplio de funciones que están disponibles por defecto en el lenguaje de programación.

El conjunto de funciones que nos ofrece un lenguaje de programación se denomina **librería**. Es importante tener en cuenta que el programador tiene la potestad de poder crear sus propias librerías y aumentar el número de funciones disponibles en el sistema. Desde el punto de vista del lenguaje de programación, el número de librerías disponibles puede ser variable.

Fig.17. Funciones nativas del lenguaje de programación.

Dependiendo del tipo de lenguaje de programación con el que estemos trabajando, podemos tener librerías más especializadas en determinados ámbitos. Por ejemplo, supongamos que estamos trabajando con un lenguaje de programación para el cálculo matricial. En este caso, el lenguaje de programación nos ofrecerá librerías especializadas para manejar matrices. Normalmente, trabajamos con lenguajes de propósito general que poseen un conjunto amplio de librerías, pero que no están sumamente especializados.

Como norma general, este tipo de librerías nos permiten realizar un amplio número de operaciones. Si nos centramos en un lenguaje en concreto, podemos analizar el conjunto de librerías que puede tener.

Los **lenguajes de programación** ofrecen un amplio número de **funciones** que están en su librería nativa. Algunas de las más relevantes son:

- **Funciones carácter**: Nos permite trabajar con caracteres. Nos ofrece funcionalidades como comprobar si un elemento tiene un determinado carácter, comprobar si un carácter es un dígito, etc.

- **Funciones numéricas**: Nos permite trabajar con amplio grupo de funciones con fines matemáticos. Por ejemplo, es posible redondear un número al entero más cercano, calcular una potencia, la raíz cuadrada, etc.

- **Funciones aleatorias**: Nos permiten realizar diferentes operaciones para obtener números pseudoaleatorios.

/ 11. Resumen y resolución del caso práctico de la unidad

El desarrollo de aplicaciones requiere la utilización de buenas prácticas para poder obtener un producto final de manera satisfactoria. De esta forma, es necesario seguir una serie de buenas prácticas para garantizar un correcto desarrollo. Una vez que generamos el producto final, es necesario que podamos incorporar mecanismos que permitan ampliar fácilmente el sistema y arreglar posibles futuros errores.

Utilizando buenas prácticas, como la programación modular, podemos garantizar un cierto nivel de reutilización del código que permite centralizar la lógica de nuestro algoritmo. Gracias a este enfoque, sería posible solucionar errores con facilidad y ampliar la funcionalidad de nuestro sistema fácilmente.

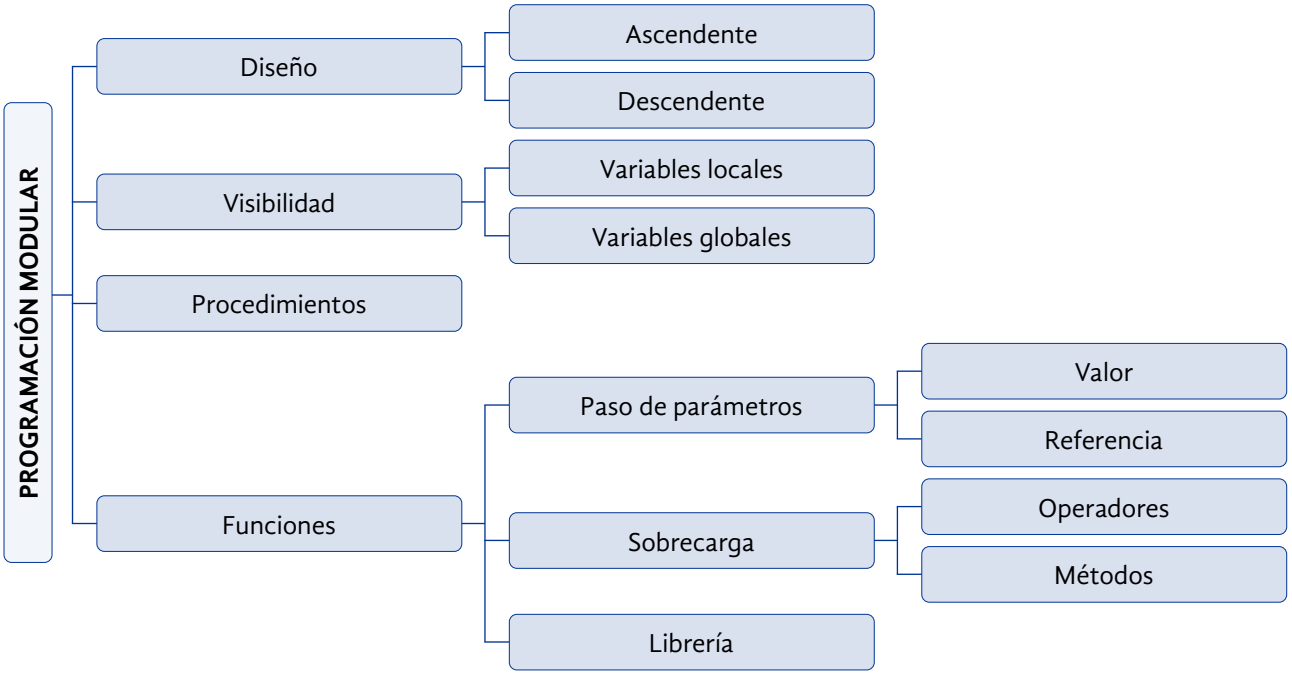

Fig.18. Esquema resumen del tema . Función Eco.

Resolución del caso práctico de la unidad

La siguiente imagen muestra una posible solución del caso práctico de la unidad.

```java
public class Eco {

    // Función que imprime el mensaje 'n' veces
    public static void eco(String mensaje, int n) {
        for (int i = 0; i < n; i++) {
            System.out.println(mensaje);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce el mensaje a imprimir");
        String mensaje = scanner.nextLine();

        System.out.print("Introduce el número de veces");
        int veces = scanner.nextInt();

        // Llamamos a la función eco para imprimir el mensaje n veces
        eco(mensaje, veces);
    }
}
```

Fig.19. Función Eco.

/ 12. Bibliografía

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Haro, C. (2021) *Algoritmia: Razonar para crear*. Ediciones Eni.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.